

Biçimsel Belirtim

(FORMAL SPECIFICATION)

Belirtim (Specification)

Bir program tasarlarken ilk bilmek isteyeceğiniz şey programın ne yapacağıdır.

Bu tanımlara spesifikasyon adı verilir.

- Müşteriden
- Son kullanıcıdan
- Analiz takımından gelebilir.

Gayriresmi yazı, yapılı belge ya da biçimsel matematiksel ifadelerle tanımlı olabilir.

Bu derste matematik notasyonu ile kesin belirtme bakacağız.

İki aracımız var: FOL ve OCL

Matematiksel Mantık

Tanımlarımızda kullandığımız akıl yürütme ve ispatları nasıl resmileştirebiliriz?

Önerme Mantığı (Propositional Logic)

- Temel mantıksal bağlaçlar
- Doğruluk tabloları
- Mantıksal denklikler

İlk-Sıra Mantık (First-Order Logic)

- Çoklu nesnelerin özellikleri hakkında akıl yürütme

Önerme (Proposition)

Doğru ya da yanlış olan ifade

- Köpek yavruları kedi yavrularından daha sevimlidir.
- Kedi yavruları köpek yavrularından daha sevimlidir.
- Bu listedeki son maddedir.

Önerme olmayanlar:

- Komutlar
- Sorular
- Anlamsız/Muğlak ifadeler

Önerme mantığında her ifade, önerme bağlaçları (connectives) ile birleştirilen önerme değişkenlerinden (variables) oluşur.

- Genellikle küçük harfler kullanılır, her değişken doğru ya da yanlıştır.

Önerme Bağlaçları

Mantıksal Değil (NOT): $\neg p$

- $\neg p$ is true if and only if p is false.
- Negation

Mantıksal Ve (AND): $p \wedge q$

- $p \wedge q$ is true if both p and q are true.
- Conjunction

Mantıksal Veya (OR): $p \vee q$

- $p \vee q$ is true if at least one of p or q are true (inclusive OR).
- Disjunction

Çıkarım (Implication): $\rightarrow$

- $p \rightarrow q$ if p is true, q is true as well.
- $\neg(p \wedge \neg q)$

İki koşullu (Biconditional): $\leftrightarrow$

- $p \leftrightarrow q$ if and only if q
- p implies q and q implies p

Doğru (True): T

Yanlış (False): $\perp$

Önerme doğruluk tabloları (FOL)

($\neg P$) // Not P

($| P Q$) // P or Q

($\& P Q$) // P and Q

($\Rightarrow P Q$) // if P then Q or P implies Q

($\Leftrightarrow P Q$) // P if and only if Q; P is equivalent to Q

P Q | ($| P Q$) ($\& P Q$) ($\neg P$) ($\Rightarrow P Q$) ($\Leftrightarrow P Q$)

P	Q	($ P Q$)	($\& P Q$)	($\neg P$)	($\Rightarrow P Q$)	($\Leftrightarrow P Q$)
T	T	T	T	F	T	T
T	F	T	F	T	F	F
F	T	T	F	T	T	F
F	F	F	F	T	T	T

Operatör Öncelikleri

$\neg \wedge \vee \rightarrow$ 

$\neg$ takip eden değişkene yapışır.

$\wedge$ ve $\vee$, $\rightarrow$ 'dan daha sıkı bağlıdır.

$\neg x \rightarrow y \vee z \rightarrow x \vee y \wedge z$

$(\neg x) \rightarrow ((y \vee z) \rightarrow (x \vee (y \wedge z)))$

Örnek Önermeler

a: Bu akşam uyanık olacağım.

b: Bu akşam ay tutulmasını göreceğim.

«Eğer bu akşam uyanık olmazsam ay tutulmasını göremem.»

$\neg a \rightarrow \neg b$

a: Bu akşam uyanık olacağım.

b: Ay tutulması göreceğim.

c: Bu akşam ay tutulması var.

«Eğer bu akşam uyanık olursam, fakat ay tutulması yoksa, ay tutulmasını göremem.»

$a \wedge \neg c \rightarrow \neg b$

De Morgan Kanunu

$$\neg(p \wedge q) \equiv \neg p \vee \neg q$$

$$\neg(p \vee q) \equiv \neg p \wedge \neg q$$

if (!p() && q()) { /* ... */ } yerine,
if (!p() || !q()) { /* ... */ } daha hızlı olur.

$$p \rightarrow q \equiv \neg(p \wedge \neg q)$$

$$\neg(p \rightarrow q) \equiv p \wedge \neg q$$

$$p \rightarrow q \equiv \neg q \rightarrow \neg p$$

Birinci Derece Mantık (First-Order Logic, FOL)

Önerme mantığındaki bağlaçları canlandırarak nesnelerin özellikleri hakkında önermeler yapmamızı sağlar.

- ifadeler (predicates) ile nesnelerin özellikleri tanımlanır.
- fonksiyonlar (functions) ile nesneler eşlenir
- niceleyiciler (quantifiers) ile çoklu nesneler için önerme kurulur.

Likes(You, Eggs) $\wedge$ Likes(You, Tomato) $\rightarrow$ Likes(You, Menemen)

- you, eggs, tomato, menemen: sabitlerdir. önerme değil, nesnelerdir
- likes: ifadedir. nesneleri argüman olarak alıp, doğru/yanlış üretirler
- $\wedge$, $\rightarrow$ bağlaçlardır.

Eşitlik

FOL'de özel bir ifade vardır: =

- İki nesne birbirine eşittir.
- Sadece nesnelere uygulanır.
- Önermeler için $\leftrightarrow$

$\text{StarOf}(\text{FavoriteMovieOf}(\text{You})) = \text{StarOf}(\text{FavoriteMovieOf}(\text{Her}))$

- StarOf ve FavoriteMovieOf fonksiyondur. Nesneleri argüman olarak alır, **sonuçta nesne üretirler**.
- $\text{ColorOf}(\text{Money})$
- $\text{MedianOf}(x, y, z)$
- $x + y$

Nesneler ve ifadeler

Nesneler (gerçek şeyler) ve ifadeleri (doğru/yanlış) ayrı tutmaya dikkat etmemiz gerekir.

Nesnelere bağlaç uygulayamayız

- $\text{Venus} \rightarrow \text{The Sun}$ ✗

Fonksiyonlara önerme uygulayamayız

- $\text{StarOf}(\text{IsRed}(\text{Sun}) \wedge \text{IsGreen}(\text{Mars}))$ ✗

Niceleyiciler (Quantifiers)

Evrensel niceleyici (Universal quantifier)

- $\forall$ «Tüm değişkenler için bu ifade doğrudur»
- $\forall n. (n \in \mathbb{N} \rightarrow (\text{Even}(n) \leftrightarrow \text{Even}(n^2)))$
- For any natural number n , n is even if n^2 is even

Varoluşsal niceleyici (Existential quantifier)

- $\exists$ «Bazı değişkenler için bu ifade doğrudur»
- $\exists x. (\text{Even}(x) \wedge \text{Prime}(x))$
- $\exists x. (\text{TallerThan}(x, \text{me}) \wedge \text{LighterThan}(x, \text{me}))$

Değişkenler ve Niceleyiciler

Her niceleyicide tanıtilan değişken ve nicelenen ifade vardır.

Değişken sadece nicelenen ifade kapsamında geçerlidir.

$(\forall x. \text{Loves}(\text{You}, x)) \rightarrow (\forall y. \text{Loves}(y, \text{You}))$

--x burada yaşar-----y burada yaşar---

$(\forall x. \text{Loves}(\text{You}, x)) \rightarrow (\forall x. \text{Loves}(x, \text{You}))$

--x burada yaşar---- başka x burada yaşar--

Değişkenler ve Niceleyiciler

$\forall x. P(x) \vee R(x) \rightarrow Q(x)$ parantezlersek;

$((\forall x. P(x)) \vee R(x)) \rightarrow Q(x)$

- Sözdizimsel olarak yanlış! x , ifadenin yarısında geçerli ve tanımlı değil!
- Parantezleri düzgün kurgulamalıyız:

$\forall x. (P(x) \vee R(x) \rightarrow Q(x))$

İfadeleri Kurgulamak

All puppies are cute.

- $\forall x. (Puppy(x) \wedge Cute(x))$
 - Elimizde puppy dışında x 'ler için de doğru olması gerekir. İngilizce'si doğru olsa bile FOL ifadesi yanlıştır.
- $\forall x. (Puppy(x) \rightarrow Cute(x))$

All P's are Q's: $\forall x. (P(x) \rightarrow Q(x))$

Some blobfish is cute.

- $\exists x. (Blobfish(x) \rightarrow Cute(x))$
 - En azından bir örnek için doğru olmalıdır. İngilizce'si doğru olsa bile FOL ifadesi yanlıştır.
- $\exists x. (Blobfish(x) \wedge Cute(x))$

Some P is a Q: $\exists x. (P(x) \wedge Q(x))$

İfadeleri Kurgulamak

Person (p), Loves(x,y)

«Everybody loves someone else.»

$\forall p. (Person(p) \rightarrow \exists q. (Person(q) \wedge p \neq q \wedge Loves(p, q)))$

For every person there is some person who isn't them that they love.

«There is a person that everyone else loves.»

$\exists p. (Person(p) \wedge \forall q. (Person(q) \wedge p \neq q \rightarrow Loves(q, p)))$

There is some person who everyone who isn't them loves.

Niceleyici Sıralama

$\forall x. \exists y. P(x, y)$

For any choice x, there's some y where P(x, y) is true.

$\exists x. \forall y. P(x, y)$

There is some x where for any choice of y, we get that P(x, y) is true.

Niceleyicileri «Değillemek» (Negation)

	Ne zaman doğru?	Ne zaman yanlış?
$\forall x. P(x)$	Tüm x'ler için $P(x)$	$\exists x. \neg P(x)$
$\exists x. P(x)$	Bazı x'ler için $P(x)$	$\forall x. \neg P(x)$
$\forall x. \neg P(x)$	Tüm x'ler için $\neg P(x)$	$\exists x. P(x)$
$\exists x. \neg P(x)$	Bazı x'ler için $\neg P(x)$	$\forall x. P(x)$

- Değil'i niceleyicinin içine itin.
- Niceleyiciyi diğeri ile değiştirin.

Niceleyicileri «Değillemek» (Negation)

$\forall x. \exists y. \text{Loves}(x, y)$

"Everyone loves someone."

$\neg \forall x. \exists y. \text{Loves}(x, y)$

$\exists x. \neg \exists y. \text{Loves}(x, y)$

$\exists x. \forall y. \neg \text{Loves}(x, y)$

"There's someone who doesn't love anyone."

Küme İşlemleri

$\text{Set}(s)$: s bir kümedir.

$x \in y$: x , y 'nin bir elemanıdır.

«Empty set exists»

$\exists S. (\text{Set}(S) \wedge \neg \exists x. x \in S)$

$\exists S. (\text{Set}(S) \wedge \forall x. \neg(x \in S))$

«Two sets are equal if and only if they contain the same elements»

$\forall S. (\text{Set}(S) \rightarrow \forall T. (\text{Set}(T) \rightarrow (S = T \iff \forall x. (x \in S \iff x \in T))))$

Küme İşlemleri

$\forall x \in S. P(x)$

"for any element x of set S , $P(x)$ holds."

$\exists x \in S. P(x)$

"there is an element x of set S where $P(x)$ holds."

Birinci Derece Mantık (First Order Logic, FOL)

Matematiksel mantık ifadeleri

- Birinci derece mantık (FOL)
- İfadesel hesap (Predicate Calculus)

Gibi isimlerle bilinir.

Önergeleri mantıksal bağlaçlarla (ve, veya, değil) kesin olarak tanımlayabilmemizi sağlar.

Niceleyicilerin (değişkenlerin) belirli tanım alanları (domain) vardır. Mantık modeli bu alan için doğru ya da yanlış olur.

$Ali(a) \Rightarrow bakkal(a)$ öyle bir a vardır ki bu a 'nın adı Alidir ve bu a bakkaldır. Bu a nasıl bir a 'dır? Bütün a 'lar için geçerli midir? Hayır. Etki alanında geçerlidir. Bütün Ali'ler bakkal olurdu.

FOL - Sözdizimi

Cümleler (önergeler) basit ve karmaşık olabilir.

İfadelerin (predicates) ismi ve terim listesi vardır. Terimler virgül ile ayrılır

- evli(Ali, Ayşe)

Terimler değişken veya sabitleri ifade eder. Değişkenler için küçük harfler kullanılır.

Karmaşık ifadeler birli, ikili ya da çoklu olabilir.

- Birli: ! ve önerme : ! sıcak(bugün)
- İkili: &, |, $\Rightarrow$, $\Leftrightarrow$ ile bağlanan iki önerme. Önergeler parantezlenir.
- Çoklu: Çoklayıcı, değişken, önerme. «all(her)» $\forall$, «exists(öyle bazı)» $\exists$. $\forall(x, \text{yaşlıdır}(x, 50))$

FOL - Sözdizimi

$\forall(x, \forall(y, bla))$ ifadesi $\forall((x,y), bla)$ olarak kısaltılabilir.

Karmaşıklık yoksa parantez kullanılmayabilir.

Önergeler için tanımlar == ile yapılabilir.

- $P == \text{yaşlı}(\text{ali}, \text{veli}) \ \& \ \text{mutlu}(\text{veli}, \text{ali})$ (P önermesi ali veliden yaşlıdır ve veli aliden mutludur)

Tanımlar değişken isimleriyle parametrik yapılabilir.

- $Q(x, y) == \forall(z, (\text{yaşlı}(x, z) \ \& \ \text{yaşlı}(z, y)) \Rightarrow \text{yaşlı}(x, y))$

$P(V \% E)$: V isim, E önerme ise, her V yerine E'yi yerleştirir. (önergelerdeki isimlere sistematik değişiklik)

FOL - Yorumlama

İfadeler (predicates) «T» ve «F» üreten fonksiyonlardır.

Tekli ve ikili operatörler kendi anlamlarını taşır.

- ! not
- & and
- | or
- $\Rightarrow$ implies
- $\Leftrightarrow$ denklik

Çoklu ifadeler, nitelenen değişkenin tanımlandığı ve sabitlendiği bir etki alanı tanımlar. Bu etki alanlarını içiçe uzatabilirsiniz.

- Değişkenlerin ismi farklı olduğu sürece etki alanı tüm iç bölgeyi kapsar.
- Aynı isimli değişkenler var ise, dıştaki içeride etki etmez, iç etki alanı bittiğinde tekrar devam eder.

OCL

UML'in bir parçası

FOL için sözdizimi sağlar.

Bu sayede UML diyagramlarına açıklama ekleyebiliriz.

Örnek: Sıralama

Kesin spesifikasyonlarımızı nasıl yaptığımızı anlayabilmek için basit bir örnek olarak sıralamayı kullanacağız.

Herkes bir listeyi nasıl sıralayabileceğini bilir.

- Bilgisayar bilmez.
- Ona sıralama programı sağlamalısınız.

Örnek: Tamsayı vektörlerini sıralayabilecek bir program için spesifikasyon yazalım.

- Girdi argümanı X , tamsayı vektörü
- Çıktı argümanı Y , X 'in artan sıralanmış hali

İlk olarak FOL kullanacağız

Sıralama

Y'nin, X'in sıralı hali olması için beklenen davranışı Türkçe tanımlayalım.

- X, tamsayı dizisi
- Y, tamsayı dizisi
- Sıralama artan şekilde

X tamsayı dizisi verildiğinde, Y tamsayı dizisini döndürürüz ve Y'deki her eleman bir sonrakinden daha küçüktür.

- Ya eşit elemanlar varsa? → küçük veya eşittir.
- Ya son eleman? Onun sonrası yok?

Girdiler/Çıktılar

İlk düşünmemiz gereken, spesifikasyon girdiyi tanımlıyor mu?

Bizde X isimli bir tamsayı vektörü denmişti.

- Böyle olmasaydı, programımızın vektör, elma, armut, kestane, muz hepsini anlamlı bir şekilde sıralaması gerekirdi.

Çıktımız da Y isimli bir tamsayı vektörü.

Sıralamanın artan yönde olmasını istiyoruz. Az önceki tanımda belirli konumlardaki değerlerin sonrakinden küçük veya eşit olmasını söyledik.

- Son eleman özel durum çünkü ondan sonra eleman yok.

Girdiye olan Duyarlılık

Önemli bir özelliğimiz daha var.

Çıktılarımızın içeriği, girdilerimizin içeriği ile birebir aynı ve tamsayı olmalıdır!

- 3, 2, 1 girdi, 4, 5, 6 çıktı diyelim. Çıktı önceki tanımımıza uyuyor?

"Çıktı Y vektöründeki her eleman, girdi X vektöründe de yer almalıdır."

- Peki ya 3, 2, 1 girdiğinde, 1, 1, 1, 1, 2, 3 çıkarsa?

"Çıktı Y vektöründeki her eleman, girdi X vektöründe aynı sayıda yer almalıdır."

- Peki ya 3, 2, 1 girdiğinde 1, 2, 3, 4 çıkarsa?

"Çıktı Y vektöründeki her eleman, girdi X vektöründe aynı sayıda yer almalıdır ve yeni eleman yer almamalıdır."

Hani kısa ve kesin tanım yapacaktık? Sıralama çok basit bir problem. Ya askeri bir yazılımda güvenlik için program yazıyorsak? Kesin ve net tanım yapabilmeliyiz.

Döngüsellik (Circularity)

Bir kavramı belirlerken, kavramın tanımda tanımlanmasına çalışırız.

Sıralamayı sıralama terimleri ile ifade etmeye çalıştık.

Bu duruma döngüsellik denir ve bizi problemi anlama konusunda ileriye taşımaz.

Tanımımıza tekrar bakalım:

Bir X tamsayı vektörü verildiğinde, Y tamsayı vektörü üretilir. Y vektörü sıralı olmalıdır (her eleman sonrakinden [küçüktür|büyüktür]). Y vektörünün içeriği, X vektörünün içeriği ile aynı olmalıdır (Y vektöründeki her eleman X'ten gelmelidir). Y'deki her bir elemanın tekrar sayısı, X'teki tekrar sayısıyla aynı olmalıdır.

Şimdi bu problemi kesin matematik dili ile modelleyelim.

Biçimsel Belirtim

Matematiksel tanım kurgusunda üç aşama vardır.

1. İmza (Signature)
2. Önkoşul (Precondition)
3. Sonkoşul (Postcondition)

İmza (Signature)

Programın adı, girdi argümanlarının adları ve tipleri, çıktılarının adları ve tipleri tanımlanır.

```
Vector<int> Y = SORT(Vector<int> X)
```

- Programın tek girdisi X, tipi tamsayı vektörü, tek çıktısı Y tamsayı vektörü.

Değişkenlere (X ve Y) gibi kesin isimler verilir. Bunlara ön ve son koşullarda atıf yapacağız. (Java'da değişken tanımlamanın aynısı)

Sıralama programımızı, "sıralamaya uygun basit veri tipi olması koşulu ile", başka veri türlerine de uygulayabiliriz.

- Sıralamaya uygunluk: >, <, >=, <=, = gibi sıralama ifadelerinde yer alabilme
- Tamsayılar, gerçek sayılar, stringler, ...

ÖnKoşullar

Önkoşullar, fonksiyonun girdi argümanları hakkındaki savlardır.

Parametreler alan bir fonksiyonu kodlarken ilk yapılan, girdilerin beklenen girdiler olup olmadığının kontrolüdür.

Biçimsel belirtimimiz ile bu kontrol koşullarının neler olduğunu belirleriz.

Başka bir örnek: `Real Y = SQRT (Real X)`

Önkoşul, X'in negatif olmamasıdır. $x \geq 0$

- Kompleks sayılarımız olsaydı, bu önkoşul olmayacaktı.
- X negatif ise olanları tanımlamıyoruz. Sadece fonksiyonun davranışını tanımlıyoruz.
- Herhangi bir girdiyi alıp, uygun olmayanlarda hata/exception üreten bir versiyon da tanımlayabilirdik.

Sıralama için Önkoşullar

Ön/son koşullarda veri tipleri ile ilgilenmeyiz. Onu imza kısmında hallettik.

X vektörü boş ise?

- Y vektörü boş döner.

$X > 0$ yapabiliriz. Ama daha genel olması için boş kümeyi de kapsamalı.

Öyleyse?

Sıralama için ön koşul yok. Ya da "True" (her koşulda çalışır)

Sonkoşullar

Fonksiyon tarafından üretilen çıktı hakkında neyin mutlaka doğru olması gerektiğini söyler.

Tipik olarak, çıktının girdi ile nasıl ilişkili olduğunu anlatır.

SQRT fonksiyonumuz için son koşul ne olabilir?

- $Y = \text{SQRT}(X)$ dersek, döngüsel tanım yaratmış oluruz.

Aralarındaki sayısal ilişkiyi tanımlamalıyız:

- $Y * Y = X$
- Bu, karekök çalıştıktan sonra doğru olması gereken ifadedir. Buna son koşul diyoruz.

Sonkoşullar

Şimdiye kadar saf fonksiyonlar ile ilgilendik.

- Saf fonksiyonlarda çıktı tamamen girdi tarafından belirlenir.

Programlama dillerinde, fonksiyon, yordam, metod gibi hesap birimleri saf olmayabilir.

- Son koşulları yazarken, çıktının girdiye nasıl bağlı olduğunun yanında, olası yan etkileri (side effect) de belirtmeliyiz.
- Yan etkiler:
 - Global değişkenlerde yapılan değişiklikler
 - Fonksiyonun girdi ya da çıktısında sonuçları gösterilmeyen işlemler
 - Nesneye yönelik dillerde çalıştığımız nesnenin bir örneğinde olan herhangi bir değişiklik

Sıralama için Sonkoşullar

Doğal dil ile nasıl tanımlamıştık, onu FOL ile nasıl gösterebiliriz?

Veri tipleri ile ilgili ifadeleri kaldırdığımızda:

Çıktı vektörü Y sıralı olmalıdır.

Y 'nin içeriği, X 'in içeriği ile aynı olmalıdır.

- X 'teki herşey Y 'de olmalıdır.
- Y 'deki herşey X 'ten gelmelidir.
- Y 'deki her bir elemanın tekrar sayısı X 'teki tekrar sayısı ile aynı olmalıdır.

Sıralama için Sonkoşullar

İki parçada halletmemiz gerekir.

İlk kısım, sıralı olması. Y tamsayı vektörünün sıralı olması ne demektir?

« Y 'deki **her bir** eleman için, **eğer** kendisinden sonra gelen bir eleman varsa, o eleman baktığımız elemandan daha büyük ya da eşit olmalıdır»

- Son eleman için hiçbir şey söylemedik.
- İfade tüm elemanlar için doğru ise, son eleman zaten en büyük eleman olmalıdır.
- Her bir tanımlayıcısı için, bir indeks değişkeni de kullanacağız. i , $i+1$

Sıralama için Sonkoşullar

İkinci kısım, eleman sayısı. Y tamsayı vektöründe kaç eleman var?

- n gibi bir değişken ile belirleyebiliriz. !! N negatif olmamalı !! i , $1..n$ arasında değer alır.
- Tanımımıza göre, $1..n-1$ arasındaki değerler için, $i+1$ 'deki elemanın büyük ya da eşit olmasını söylüyoruz.
- $N=0$ ise?? $N-1$ negatiftir?? $i:=1..n-1$, n 0 olduğu için aslında hiç «i» olmayacak. Saçma ama ifade doğru ☺ Son koşulun doğruluğunu değiştirmiyor.

$\forall i: 1 \leq i \leq (n-1), y[i] \leq y[i+1]$

Bu y 'ler nelerdir ve girdiye nasıl bağlıdır söylemedik. Sonra bunları da belirteceğiz.

OCL de aslında FOL üzerinde yer alan farklı bir sözdizimidir. Burada çok az değinip, sonra FOL gibi bir ders yapacağız.

Sıralı için önkoşul?

Bool $Y = \text{ORDERED}(\text{Vector} \langle \text{int} \rangle X)$

Tip kontrolü yapmıyoruz, bu nedenle hangi X gelirse gelsin sıralı mı diye sorabiliriz.

X boş bir vektörse?

- Amacımız kodlamaya yol göstermek. Kullanıcı karşılaşmadan durumu ele alabilmek.
- Doğal olarak sıralıdır diyebiliriz. İfademiz de boş vektör için geçerlidir.

Son koşul nedir?

Sıralı için Sonkoşul

$\forall i: 1 \leq i < |X| \bullet X[i] \leq X[i+1]$

Y'deki baştan sondan bir öncekine kadar tüm pozisyonlar için, vektörün o pozisyonunda saklanan değer, sonraki pozisyonda saklanandan büyük değildir.

$\forall$: belirteç

i: indeks değişkeni

|...|: vektörün boyutu

•: Durum mutlaka böyle olmalı (it must be the case)

TersSıralı için İmza, ilkkoşul, Sonkoşul?

Bool Y = TersSıralı (Vector <int> X)

Önkoşul: True

Sonkoşul: $\forall i: 1 \leq i < |X| \bullet X[i] \geq X[i+1]$

Sıralama için Sonkoşullar

Sıralı olması kısmını hallettik.

Şimdi, ikinci kısım: Y'nin X ile aynı elemanlardan oluşması (Ve tabii ki aynı sayıda tekrar içermesi).

İmza: $\text{Bool } Z = \text{AynıElemanlardanOluşur}(\text{Vector } \langle \text{int} \rangle X, \text{Vector } \langle \text{int} \rangle Y)$

Önkoşul: İki vektörün aynı uzunlukta olması $|X| = |Y|$

Sonkoşul: Üç ifademizi de FOL ile tanımlamalıyız:

- X'teki herşey Y'de olmalıdır.
- Y'deki herşey X'ten gelmelidir.
- Y'deki her bir elemanın tekrar sayısı X'teki tekrar sayısı ile aynı olmalıdır.

Ya da, ...

Permütasyon

Permütasyonun tanımı: X'in elemanları, Y'nin elemanlarının bir permütasyonu ise; X, Y ile aynı elemanlara sahiptir.

Permütasyon iyi tanımlanmış bir matematiksel özellik olduğu için, kendimiz yazmak zorunda kalmayız.

$\text{Bool } Z = \text{Permütasyon}(\text{Vector } \langle \text{int} \rangle X, \text{Vector } \langle \text{int} \rangle Y)$

Son koşulları biraz hileli. Özel durumlara bölüp her durumu ayrı ele alırız.

- 1. durum: iki vektör de boş: $|X|=0 \Rightarrow \text{Permütasyon}(X, Y)$
- 2. durum: Vektörler boş değil. İlk elemanları aynı mı değil mi?
- 2.1: Aynı: X ve Y'nin kalan kısımları permütasyonu mu? $|X| > 0 \wedge X[1] = Y[1] \wedge \text{Permütasyon}(\text{kalan}(X), \text{kalan}(Y)) \Rightarrow \text{Permütasyon}(X, Y)$ (özyinelemeli. Kalan her zaman daha küçük ve sıfır uzunluk durumunu ele aldık.)

Permütasyon

- 2.2: İlk elemanları Farklı: Y vektörünü, X[1]'in geçtiği yere göre üç parçaya böleriz.
- Ör. X[1]=5 ise, Y'nin bir yerinde 5 olmalı. Yoksa, permütasyonu değildir.
- J. Pozisyonda olsun, $j > 1$ ve $j \leq |Y|$ olmalı.
- Y'nin birinci parçası, 1.den j.ye (j dahil değil),
- Y'nin ikinci parçası, j. Eleman
- Y'nin üçüncü parçası, j+1'de sona kadar kalan elemanlar

Üçüne de bakarak permütasyon kararını veririz.

$\exists j: 1 < j \leq |Y| \bullet (X[1] = Y[j])$

Kalan iki (1. ve 3.) parça için ifadeleri yapıştırırız: $Y[1..j-1] \circ Y[j+1..|Y|]$ (concat)

Permütasyon(kalan(X), $(Y[1..j-1] \circ Y[j+1..|Y|])$)

X'in kalan kısmı ile, Y'nin j.pozisyon harici kısmı birbirinin permütasyonu mu? Yine rekürsif. Boyutları azalttığımız için, yine tüme varabiliriz.

Permütasyon için Sonkoşul

Parça parça ilerledik, şimdi:

Permütasyon için sonkoşul:

Permütasyon(X,Y) $\Leftrightarrow$

$|X|=0 \vee$

$(|X| > 0 \Rightarrow$

$(X[1] = Y[1] \wedge \text{Permütasyon}(\text{kalan}(X), \text{kalan}(Y))) \vee$

$(X[1] \neq Y[1] \wedge$

$(\exists j: 1 < j \leq |Y| \bullet (X[1] = Y[j])) \wedge$

$\text{Permütasyon}(\text{kalan}(X), (Y[1..j-1] \circ Y[j+1..|Y|])))) \vee$

Kontrol edelim

X boş vektör ise belirtim ne söylüyor?

- Y de boş vektördür, ikisi aynı elemanlara sahiptir.

X'in ilk elemanı, Y'de birden fazla yerde geçerse?

- Tanımımız ile bunu da karşılayabiliriz.
- Geçtiği yerleri bulabiliyoruz. Rekürsif olarak kalan parçalara bakıyoruz. Her bulduğumuzda yeni özyineleme yaratırız. Boş vektör karşılaştırmasına kadar ineriz.

OCL

Buraya kadar FOL ile belirtim yaptık.

UML'de FOL'un bir başka hali olan OCL bulunur.

Sıralı() için OCL belirtimi:

context Vector::Sıralı() : Boolean

- Metodun argümanı yok. Tüm nesneye dayalı dillerdeki gibi mesajın yollandığı nesne belirtilen argüman olarak hizmet eder.

pre: true

- True ise tanımlamaya gerek yok.

post: self -> forAll (i:Integer | i > 0

and i < self.length implies

self.at[i] <= self.at[i+1]

OCL - FOL farkları:

| sembolü değişkeni önermeden ayırmak için kullanılır.

«i» değişkeni için kısıtlar önermenin parçası olarak yer alır.

«i» değişkeni için kısıtları «implies» anahtar kelimesi ile belirtir.

OCL'de vector yoktur, sequence vardır, ama aslında ikisi de aynı şeydir.

Sonuç

Bazen, bir sistemin gerektirdiği işlevselliği kesin olarak tanımlamamız gerekir.

Bu tanımlamaları yapmak için çeşitli biçimsel diller ve araçlar bulunmaktadır.

Bunları kullanabilmek için, tam olarak ne söylemeye çalıştığınız hakkında çok sıkı düşünmeniz gereklidir.

Bu efor, sizi anlaşılmayan gereksinimler yüzünden çokça tekrar çalışmadan kurtarabilir.